

**38-meteor-dodge-...**
File**Lab_4_VibeCoding...**
PDF

lets start and complete the assignment im done with repo and
beggining video

Let's complete Lab 4 step by step using **ChatGPT only**, with short, natural prompts. We'll work on your existing Meteor Dodge repository, fix the bugs, implement the features listed in its README, test everything, and push separate commits for each task.

Your handout specifies individual work, a 10-second video before changes, another after all changes, separate commits for each task, and three deliverables: videos, updated code, and exported chat history.

Lab_4_VibeCoding_Student_handout.pdf

Our workflow



Step 1 — Initial setup

Repository cloned, game running, and before-video recording started.



Step 2 — Fix the first bug

Inspect the code, give ChatGPT a focused prompt, test the fix, and commit it.



Step 3 — Implement required features

Read the README and handle each feature individually, with its own commit.



Step 4 — Final testing

Check the fixes and features, then record the after-video.



Step 5 — Submit

Push to your personal repository and prepare the Lab-4 deliverables.

Step 1 — Finish your before-video

The README confirms the exact tasks we need to complete: one bug fix and three features. Source: repository README.

1. Record around 10 seconds of the original game.

2. Show the title screen and launch the game with **Space**.
3. During active flight, press **Space** again to demonstrate that the laser doesn't fire.
4. Save the recording without modifying the code.

Step 2 — Start with the first bug

Once your recording is saved, send this short prompt to ChatGPT:

Writing



Inspect my Meteor Dodge code and find why Space launches the game but doesn't fire lasers during flight. Fix only this bug, keep the existing structure, and explain which files changed and how to test it. Don't implement other features yet.

Follow the suggested fix, inspect the changed code, and run the game again. We won't move to the next task until this bug works.

Tell me once the before-video is saved. Then we'll work through the first bug together, one step at a time.

I'll give prompts u give code

Done. Send me each prompt one by one. I'll provide the **exact code changes** needed for your Meteor Dodge repository, using ChatGPT only.

For every prompt, I'll:

1. Identify the relevant file(s).
2. Give you the complete corrected code or precise changes.
3. Tell you how to test the result.
4. Give you a separate Git commit message.

I'll stick to the requested task, preserve existing code where possible, and avoid implementing future features prematurely.

Send your first prompt.

Fix the laser bug. Space should launch or restart the game on the title/game-over screen, but fire a laser during active gameplay.

Inspect the existing code and make the smallest necessary changes.
Don't add any other features. Explain how I can test the fix.

Worked for 31s

The bug is in `game/game_engine.py`. The current Space-key handler only starts or restarts the game; it never creates a laser during active gameplay.

1. Make the smallest necessary change

In `game/game_engine.py`, replace the existing `handle_events()` method with this:

</> Python

```
def handle_events(self):
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            return False
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_SPACE:
                if self.game_over:
                    self.reset()
                elif not self.started:
                    self.started = True
                else:
                    self.ship.fire()
    return True
```

Important: This change assumes `Ship` has a `fire()` method, but the current `game/ship.py` does not. We therefore need a small addition to `Ship` and a laser update/drawing implementation for the bug to be fully fixed.

2. Add laser support in `game/ship.py`

The existing ship class has no laser logic. Add this method inside the `Ship` class:

</> Python

```
def fire(self):
    if not hasattr(self, "lasers"):
        self.lasers = []
```

```
self.lasers.append(
    pygame.Rect(self.rect.centerx - 2, self.rect.top - 12, 4, 12)
)
```

Then initialize the laser list in `__init__`:

</> Python

```
self.lasers = []
```

3. Update and draw lasers

In `game/game_engine.py`, add the following inside `update()`, after the ship movement:

</> Python

```
for laser in self.ship.lasers:
    laser.y -= 8
self.ship.lasers = [
    laser for laser in self.ship.lasers if laser.bottom > 0
]
```

In `draw()`, after drawing the ship, add:

</> Python

```
for laser in self.ship.lasers:
    pygame.draw.rect(self.screen, (80, 255, 255), laser)
```

These changes allow Space to launch/restart on the relevant screens and fire a visible laser while flying.

4. Test the fix

1. Run `python main.py`.
2. Press Space on the title screen — the game should launch.
3. Press Space during flight — a cyan laser should appear above the ship.

4. Move using WASD or arrow keys and fire again.
5. Let a meteor hit the ship, then press Space — the game should restart.

One limitation: This implements visible laser firing, but it does not yet make lasers destroy meteors. That is a separate part of the laser mechanic and should be added if the task requires the project's full expected behavior.

Commit the changes with:

</> Bash

```
git add game/game_engine.py game/ship.py
git commit -m "Fix laser firing input bug"
```

Implement meteor splitting. When a large meteor is destroyed by a laser, split it into smaller fragments that move outward. Small meteors should disappear completely. Keep the existing game mechanics unchanged.

Worked for 13s

Make the changes in `game/meteor.py` and `game/game_engine.py`. This keeps the existing movement and collision mechanics intact while adding splitting when a laser destroys a meteor.

1. Update `game/meteor.py`

Add `fragment()` inside the `Meteor` class. It returns two smaller meteors for a large meteor, or no fragments for a small one.

</> Python

```
def fragment(self):
    if self.radius < 20:
        return []

    fragments = []
    for angle in (45, 135):
        meteor = Meteor.__new__(Meteor)
        meteor.x = self.x
        meteor.y = self.y
```

```

meteor.radius = max(8, self.radius // 2)
meteor.vx = self.vx + random.uniform(-2, 2)
meteor.vy = self.vy + random.uniform(-1, 1)
meteor.color = self.color
meteor.rot = self.rot
meteor.rot_speed = random.uniform(-3, 3)

speed = 3
meteor.vx += speed * math.cos(math.radians(angle))
meteor.vy += speed * math.sin(math.radians(angle))

```

2. Update game/game_engine.py

Inside `update()`, add laser collision handling after `self.ship.move(...)`. This assumes your previous laser implementation stores laser rectangles in `self.ship.lasers`.

</> Python

```

remaining_lasers = []
for laser in self.ship.lasers:
    hit = False
    for meteor in self.meteors[:]:
        dx = meteor.x - laser.centerx
        dy = meteor.y - laser.centery
        if dx * dx + dy * dy < meteor.radius * meteor.radius:
            self.meteors.remove(meteor)
            self.meteors.extend(meteor.fragment())
            hit = True
            break

    if not hit:
        laser.y -= 8
        if laser.bottom > 0:
            remaining_lasers.append(laser)

self.ship.lasers = remaining_lasers

```

Important: If your current `update()` already moves lasers, remove that duplicate movement logic so each laser moves only once per frame.

3. Test it

- Fire at a large meteor: it should split into two smaller fragments moving apart.
- Fire at a small meteor: it should disappear without splitting.
- Confirm that ship movement, meteor spawning, collision with the ship, and restart still work.

Commit this task separately:

</> Bash

```
git add game/meteor.py game/game_engine.py
git commit -m "Add meteor splitting on laser hits"
```

Add collectible shield power-up orbs. Make them drift through the game so the ship can collect them. A collected orb should give a temporary shield that absorbs exactly one meteor collision. Show the shield clearly and remove it after it absorbs a hit or expires.

Worked for 9s

Make these changes in `game/game_engine.py` and `game/ship.py`. This adds drifting collectible orbs and a visible temporary shield without changing the existing movement or meteor-spawning mechanics.

1. Update `game/game_engine.py`

A. Add these variables inside `reset()`, alongside the existing game-state variables:

</> Python

```
self.powerups = []
self.shield_active = False
self.shield_timer = 0
self.powerup_spawn_timer = 0
```

B. Add this method inside `GameEngine` to spawn and draw the orbs:

</> Python

```
def spawn_powerup(self):
    orb = pygame.Rect(
```

```
        random.randint(15, WIDTH - 15), -20, 20, 20
    )
    self.powerups.append(orb)
```

C. Inside `update()`, after `self.ship.move(keys, WIDTH, HEIGHT)`, add:

</> Python

```
# Spawn drifting shield orbs periodically
self.powerup_spawn_timer += 1
if self.powerup_spawn_timer >= 600:
    self.spawn_powerup()
    self.powerup_spawn_timer = 0

# Move orbs and collect them
for orb in self.powerups:
    orb.y += 2

self.powerups = [
    orb for orb in self.powerups if orb.top < HEIGHT
]

for orb in self.powerups[:]:
    if self.ship.rect.colliderect(orb):
        self.powerups.remove(orb)
        self.shield_active = True
        self.shield_timer = FPS * 8
```

D. Replace the existing meteor collision block:

</> Python

```
for m in self.meteors:
    m.update()
    if m.collides(self.ship.rect):
        self.game_over=True
```

with:

</> Python

```
for m in self.meteors:
    m.update()
    if m.collides(self.ship.rect):
        if self.shield_active:
            self.shield_active = False
            self.shield_timer = 0
            self.meteors.remove(m)
            break
        else:
            self.game_over = True
            break
```

E. In `draw()`, after `self.ship.draw(self.screen)`, add:

</> Python

```
# Draw collectible shield orbs
for orb in self.powerups:
    pygame.draw.circle(self.screen, (50, 255, 220), orb.center, 10)
    pygame.draw.circle(self.screen, (200, 255, 255), orb.center, 5)

# Draw the active shield around the ship
if self.shield_active:
    pygame.draw.circle(
        self.screen, (50, 255, 220),
        self.ship.rect.center, 30, 3
    )
    shield_text = self.font.render("SHIELD", True, (50, 255, 220))
    self.screen.blit(shield_text, (10, 42))
```

2. Update game/ship.py

No changes are needed. The shield is managed by the game engine and rendered around the existing ship.

3. Test the feature

1. Launch the game and wait for an orb to spawn.
2. Collect the cyan orb with the ship.
3. Confirm a visible shield appears.
4. Collide with a meteor: the shield should absorb that collision and disappear, while the game continues.
5. Collect another orb and avoid collisions for eight seconds; the shield should expire.
6. Confirm that a collision without a shield still ends the game.

Commit this feature separately:

</> Bash

```
git add game/game_engine.py game/ship.py
git commit -m "Add collectible temporary shield power-ups"
```

Note: This implementation uses a fixed spawn interval of 10 seconds and an 8-second shield duration. If you want the orb to spawn more frequently for easier testing, temporarily change `600` to `180` (3 seconds at 60 FPS).

Add a score multiplier that increases every 10 seconds of continuous survival without taking a hit. Apply the multiplier to score gain and reset the streak when the player takes a hit. Display the current multiplier on the HUD. Keep the existing scoring and game mechanics consistent.

Make the changes in `game/game_engine.py` only. The existing game uses `self.score += 1` per frame, so we'll preserve that scoring rate and apply a multiplier after every 10 seconds of survival.

1. Initialize the streak

Inside `reset()`, immediately after `self.score=0`, add:

</> Python

```
self.survival_streak = 0
self.multiplier = 1
```

2. Update the collision and scoring logic

In `update()`, replace the existing meteor collision loop and score increment with this code:

</> Python

```
for m in self.meteors:
    m.update()
    if m.collides(self.ship.rect):
        self.game_over = True
        self.survival_streak = 0
        self.multiplier = 1
        break

self.meteors = [
    m for m in self.meteors if not m.off_screen(HEIGHT)
]

if not self.game_over:
    self.survival_streak += 1
    self.multiplier = 1 + self.survival_streak // (FPS * 10)
    self.score += self.multiplier
```

This means the multiplier progresses from $\times 1$ to $\times 2$ after 10 seconds, $\times 3$ after 20 seconds, and so on. Taking a meteor hit resets the streak and multiplier to $\times 1$.

3. Display the multiplier on the HUD

In `draw()`, immediately after the existing Time HUD rendering lines, add:

</> Python

```
mult = self.font.render(
    f"Multiplier: x{self.multiplier}", True, (255, 210, 80)
)
self.screen.blit(mult, (10, 42))
```

4. Test it

1. Launch the game and survive for 10 seconds; verify the HUD changes to Multiplier: $\times 2$.

2. Survive for 20 seconds; verify it reaches $\times 3$.
3. Take a meteor hit; verify the game-over screen shows the multiplier reset to $\times 1$.
4. Restart and verify the streak begins again at $\times 1$.

Commit this task separately:

</> Bash

```
git add game/game_engine.py
git commit -m "Add survival score multiplier"
```

Note: This follows the original game's existing collision behavior. If your shield power-up has already been integrated, a shield-absorbed collision should not reset the streak; in that case, the streak reset belongs only in the unshielded-hit branch.

Add a score multiplier that increases every 10 seconds of continuous survival without taking a hit. Apply the multiplier to score gain and reset the streak when the player takes a hit. Display the current multiplier on the HUD. Keep the existing scoring and game mechanics consistent.

We'll implement this in `game/game_engine.py` only. One important detail: the existing game uses `self.score` to display survival time, so multiplying it directly would make the displayed time inaccurate. We'll keep survival time separate and apply the multiplier to score gain.

1. Initialize the multiplier

Inside `reset()`, after `self.score=0`, add:

</> Python

```
self.survival_frames = 0
self.survival_streak = 0
self.multiplier = 1
```

2. Update scoring and survival tracking

In `update()`, replace the existing meteor collision loop and score increment with:

</> Python

```

hit = False

for m in self.meteors:
    m.update()
    if m.collides(self.ship.rect):
        self.game_over = True
        hit = True
        break

self.meteors = [
    m for m in self.meteors if not m.off_screen(HEIGHT)
]

if hit:
    self.survival_streak = 0
    self.multiplier = 1
else:
    self.survival_frames += 1
    self.survival_streak += 1
    self.multiplier = 1 + self.survival_streak // (FPS * 10)

```

This preserves the original meteor collision behavior and increases the multiplier every 10 seconds of survival.

3. Keep the survival-time display accurate

In `draw()`, replace:

</> Python

```
sc=self.font.render(f"Time: {self.score//60}s",True,(200,200,240))
```

with:

</> Python

```

sc = self.font.render(
    f"Time: {self.survival_frames // FPS}s",

```

```
True,  
(200, 200, 240)  
)
```

Also replace the game-over message's `self.score//60` with `self.survival_frames//FPS`.

4. Display the multiplier

Immediately after the existing time HUD rendering lines, add:

</> Python

```
mult_text = self.font.render(  
    f"Multiplier: x{self.multiplier}",  
    True,  
    (255, 210, 80)  
)  
self.screen.blit(mult_text, (10, 42))
```

5. Test and commit

- Survive 10 seconds: multiplier becomes $\times 2$.
- Survive 20 seconds: multiplier becomes $\times 3$.
- Collide with a meteor: multiplier resets to $\times 1$.
- Restart: score and survival streak start fresh.
- Verify the survival-time display still reflects real elapsed gameplay time.

</> Bash

```
git add game/game_engine.py  
git commit -m "Add survival score multiplier"
```

Shield compatibility: If you've already integrated the shield feature, a meteor collision absorbed by the shield should not reset the streak. Reset it only when a collision actually ends the game.

Review all four completed tasks against the README. Check for bugs, broken controls, collision issues, and incorrect resets. Fix only

genuine problems, preserve all required features, and tell me exactly how to test the final game. Don't add new features.

Worked for 54s

I reviewed the uploaded project against its README and prepared a corrected, integrated version of Meteor Dodge.

Important finding: The ZIP you provided contains the original starter code, not the four completed implementations. I therefore integrated all four required tasks into a clean copy rather than claiming to review changes that weren't present.



Meteor Dodge — reviewed code

Python / Pygame · All four tasks integrated

[Download the corrected project ZIP ↗](#)

1. What I corrected

The implementation preserves the original ship controls, randomized meteor spawning, difficulty scaling, and game-over/restart flow specified in the README.  [Lab_4_VibeCoding_Student_handout.pdf](#)

Task	Final behavior
Laser firing	Space launches or restarts the game on the appropriate screen and fires during active gameplay.
Meteor splitting	Large meteors split into two outward-moving fragments; small meteors disappear when hit.
Shield power-up	Drifting orbs grant a visible shield lasting up to eight seconds, which absorbs one collision.
Survival multiplier	The score multiplier increases every 10 seconds and resets after a meteor hit.

I also addressed several integration problems:

- Lasers move only once per frame and are removed when they leave the screen or hit a meteor.
- Meteor lists are modified safely during collision processing.
- Restart resets the score, timers, meteors, lasers, power-ups, shield, and multiplier.
- Survival time is tracked separately from score, so multiplying score does not distort the elapsed-time display.
- A shield-absorbed collision resets the uninterrupted-survival streak.

2. Verification status

 Python syntax check passed

All four Python files compiled without syntax errors.

 Runtime testing still required

The test environment does not have Pygame installed, so I could not verify actual gameplay or collision behavior by running the game.

3. Exactly how to test the final game

Extract the downloaded ZIP, open a terminal in the project folder, and run:

 **Bash**

```
pip install -r requirements.txt
python main.py
```

Then follow this checklist.

Final test checklist

0/19 checked

1. Controls and launch

- ☐ Space launches the game from the title screen.
- ☐ WASD and arrow keys move the ship in all four directions.
- ☐ The ship stays within the window boundaries.
- ☐ Space fires visible lasers during gameplay.

2. Laser and meteor collisions

- ☐ A laser hit destroys a small meteor without fragments.
- ☐ A laser hit on a large meteor produces two smaller, diverging fragments.
- ☐ Meteors still spawn and move with varying trajectories.
- ☐ An unshielded meteor collision ends the game.

3. Shield power-up

- ☐ A cyan orb appears and drifts down the screen.
- ☐ Collecting the orb displays a shield around the ship.
- ☐ The first meteor collision consumes the shield and the run continues.
- ☐ A second collision without another shield ends the run.
- ☐ A shield expires after eight seconds if unused.

4. Scoring and restart

- ☐ The multiplier starts at $\times 1$.
- ☐ After 10 seconds without a hit, the multiplier reaches $\times 2$; after 20 seconds, $\times 3$.
- ☐ A meteor hit resets the multiplier and survival streak.
- ☐ Score gain follows the current multiplier.
- ☐ The game-over screen displays the correct survival time.
- ☐ Pressing Space restarts with clean state and no leftover objects or shield.

Submission reminder: Before you submit, ensure the final video demonstrates the working features and your Lab-4 repository contains the updated code and complete chat history. Your assignment also requires separate commits for each task, so keep the four task commits separate when transferring the changes into your personal GitHub repository.

Explain the changes made for each of the four tasks in simple student-friendly language. For each task, mention the files changed, the bug or feature addressed, and how I tested it. Don't claim tests passed unless I actually ran them.

Worked for 11s

Here is a simple, student-friendly explanation of all four tasks implemented in the reviewed Meteor Dodge project. You can use this as a reference when preparing your lab submission or explaining your code to your faculty.

Testing status: I have not run the game interactively, so the test cases below are steps for you to perform, not claims that the features have passed testing.

Task 1 — Fix the laser firing bug

File changed: `game/game_engine.py`, `game/ship.py`

Problem: Pressing Space launched the game, but did nothing during active gameplay. The ship did not have a laser-firing method.

Changes made:

- Added a `fire()` method to the `Ship` class.
- Created a laser projectile in front of the ship whenever `fire()` is called.
- Updated the Space-key event handler to distinguish between the title screen, active gameplay, and game-over screen.
- Moved lasers upward and removed them when they leave the screen or hit a meteor.
- Drew lasers in cyan so they are visible.

How to test it manually:

1. Run `python main.py`.
2. Press Space on the title screen. The game should start.
3. Press Space during gameplay. A cyan laser should appear and travel upward.
4. Fire at a meteor and check that the laser destroys it.
5. Allow the ship to be destroyed and press Space to restart.

Task 2 — Implement meteor splitting

Files changed: `game/meteor.py`, `game/game_engine.py`

Problem: Large meteors disappeared completely when destroyed. They did not break into smaller fragments.

Changes made:

- Added a `fragment()` method to the `Meteor` class.
- Large meteors create two smaller meteors when hit by a laser.
- The fragments move outward in different horizontal directions.
- Small meteors disappear completely instead of splitting again.
- Updated laser collision handling to remove the original meteor and add its fragments to the meteor list.

How to test it manually:

1. Start the game and fire lasers at large meteors.
2. Check whether a large meteor breaks into two smaller fragments.
3. Observe whether the fragments move apart.
4. Fire at a small meteor and check that it disappears without creating more fragments.
5. Check that normal meteor spawning and ship collisions still work.

Task 3 — Implement collectible shield power-ups

Files changed: `game/game_engine.py` , `game/ship.py`

Problem: A meteor collision immediately ended the game, leaving the player with no protection.

Changes made:

- Added collectible cyan power-up orbs that spawn periodically and drift downward.
- Detecting a collision between the ship and an orb activates the shield.
- Displayed a cyan circle around the ship and a `SHIELD` label when protection is active.
- Set the shield duration to eight seconds.
- Made the shield absorb one meteor collision before disappearing.
- Ensured an unused shield expires after its timer runs out.
- Reset the shield and power-up state when restarting the game.

How to test it manually:

1. Launch the game and wait for an orb to appear.
2. Move the ship into the orb.
3. Check that the shield indicator appears.
4. Collide with a meteor and verify that the shield disappears without ending the game.
5. Collect another orb and avoid collisions for eight seconds. The shield should expire.
6. Without a shield, collide with a meteor and check that the game ends normally.

Task 4 — Implement the survival score multiplier

File changed: `game/game_engine.py`

Problem: The score increased at a fixed rate regardless of how long the player survived without a hit.

Changes made:

- Added variables to track survival frames, the uninterrupted survival streak, and the current multiplier.
- Increased the multiplier every 10 seconds of continuous survival.
- Applied the multiplier to score gain.
- Reset the streak and multiplier after a meteor hit.
- Added the current multiplier to the HUD.
- Kept survival time separate from score so the displayed elapsed time remains accurate.

How to test it manually:

1. Start a new game and check that the multiplier starts at $\times 1$.
2. Survive for 10 seconds without taking a hit. The multiplier should become $\times 2$.
3. Survive for 20 seconds without taking a hit. It should become $\times 3$.

4. Get hit by a meteor and check that the multiplier resets to $\times 1$.
5. Restart the game and verify that the score, timer, and multiplier reset.

Final file summary

File	Purpose
game/game_engine.py	Handles game states, keyboard input, lasers, collisions, power-ups, scoring, HUD, and resets.
game/ship.py	Adds the ship's laser-firing method and stores its laser projectiles.
game/meteor.py	Adds meteor-fragment creation and outward movement for split meteors.

The main integration goal was to make all four tasks work together while preserving the original movement controls, meteor spawning, and restart functionality.

Before submitting, run the game and perform the manual checks above. Record your after-video only after you have verified the features, and keep a separate Git commit for each of the four tasks as required by the lab instructions.